

Microprocessor Systems - CSU23021

Assignment #02 –Game to Teach Morse Code

Group 2 - Arijita Aggarwal · Marcel Czubak · Shipra Singh · William Blanco Morrissey

1. Introduction

This document is the project report for the CSU23021 Group 2 Morse Code Game, developed as Assignment #02 for the Microprocessor Systems module at Trinity College Dublin. The task was to build an interactive embedded application on the Raspberry Pi Pico (RP2040) MAKER-PI-PICO board that teaches players Morse code through structured gameplay, combining C and ARM Cortex-M0+ assembly language.

The application accepts player input entirely through the GP21 push-button. A press shorter than 180 ms is recorded as a Morse dot; a press of 180 ms or longer as a dash. When no further presses are detected for at least 800 ms after a release, the assembled sequence is committed and passed to the C game logic for decoding and evaluation. All button edge detection, timestamp recording, and Morse buffer management are performed by the ARM assembly module (assign02.S). Game logic, display, LED control, and buzzer output are handled in C.

The application implements four progressive game levels:

- Level 1 – Individual characters with their Morse code hint displayed.
- Level 2 – Individual characters without any Morse hint.
- Level 3 – Complete English words with the full Morse sequence displayed.
- Level 4 – Complete English words with no Morse hint.

At startup, the player chooses a difficulty via Morse input: Easy (one dot, 'E') starts from Level 1; Hard (four dots, 'H') starts from Level 3. Players begin with three lives. A correct answer adds one life (maximum three); an incorrect answer removes one. Five consecutive correct answers advance the player to the next level. The WS2812 RGB LED on GP28 (PIO-driven) reflects remaining lives: green for three, yellow for two, red for one, and blue when idle. The RP2040 Watchdog Timer resets the board after ~8.3 seconds of inactivity. Two innovative features are implemented: a real-time console press-duration bar chart (Feature #1) and a PWM buzzer (Feature #2).

2. Code Implementation

2.1 Architecture and File Structure

File	Language	Role
assign02.c	C	Entry point. Initialises PIO, buzzer, and GPIO. Runs the top-level game loop.
assign02.S	ARM Assembly	GPIO ISR, gap-detection (check_space), all shared timing state.
game.c / game.h	C	Morse table, character/word pools, all-around and level functions. Contains both innovative features.
welcome.c / welcome.h	C	Startup banner with animated typing effect.
ws2812.pio	PIO Assembly	WS2812 protocol: 24-bit GRB serial at 800 kHz.

2.2 ARM Assembly – assign02.S

2.2.1 init_gpio_asm

Called once from main() after PIO initialisation. Calls install_gpio_isr (patches the gpio_isr address into the RAM vector table at offset 0x74, then sets NVIC_ICPR and NVIC_ISER for interrupt #13), init_btn (configures GP21 as input with both-edge IRQ), and init_led (configures GP25 as output for a press indicator).

2.2.2 gpio_isr – Interrupt Service Routine

Fires on both GP21 edges. Reads PROC0_INTS2 to determine which edge triggered, clears the flag via INTR2, then branches:

- **Falling edge (press):** records TIMER_TIMELR to time_pressed; resets time_released and space_printed to zero to begin a new symbol.
- **Rising edge (release):** records release timestamp; computes press_duration = time_released – time_pressed. Appends '.' if press_duration < 180 000 μ s, '-' otherwise. Exports press_duration as .global for the C visual timeline (Feature #1).

All timing state lives in the .data section. morse_input_buffer and morse_input_ready are declared extern and defined as volatile in game.c.

2.2.3 check_space – Gap Detection

Polled from C, not from within the ISR. Computes (current_time – time_released) and compares to SPACE_TIME (800 000 μ s). When exceeded, null-terminates the buffer, copies it byte-by-byte into morse_input_buffer, sets morse_input_ready = 1, resets buffer_len, and sets space_printed = 1.

2.3 C Implementation – assign02.c

2.3.1 WS2812 RGB LED

PIO0/SM0 is configured via ws2812_program_init for 800 kHz. set_rgb(color) pushes a 24-bit GRB value to the TX FIFO. Due to WS2812's GRB byte order: 0x1F0000 = green, 0x001F00 = red, 0x00001F = blue. update_lives_led(lives) maps 3 lives $\rightarrow$ green, 2 $\rightarrow$ yellow (0x1F1F00), 1 $\rightarrow$ red. set_idle_color() sends blue at startup.

The four LED states as observed on the physical hardware are shown below:

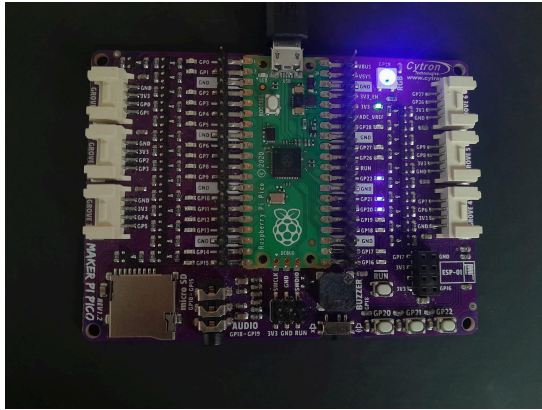


Figure 1 – Blue LED: idle state (game not started)

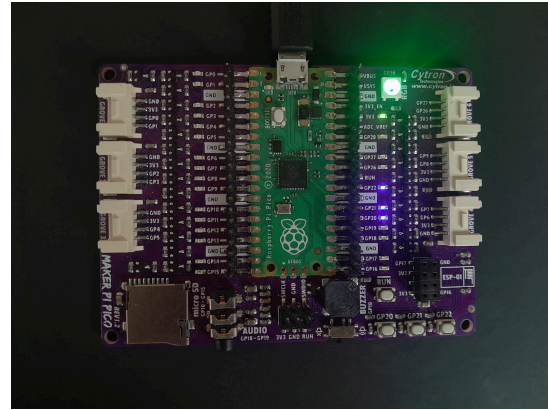


Figure 1 – Green LED: 3 lives (full health)

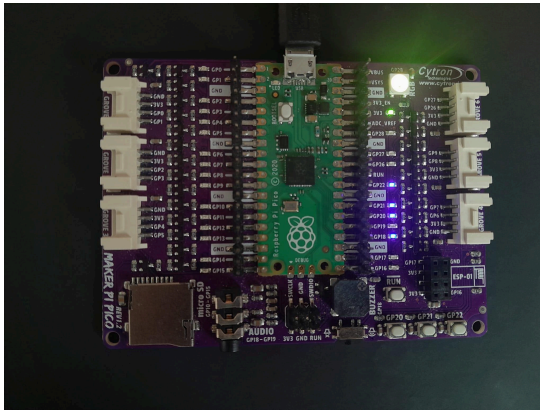


Figure 3 – Green LED: active gameplay/1 life remaining

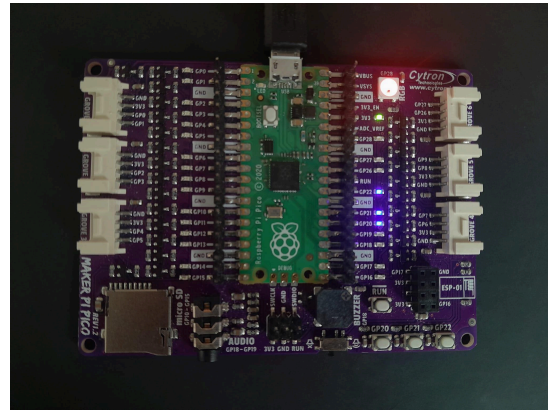


Figure 4 – Red LED: game over

2.3.2 PWM Buzzer Initialisation

buzzer_init() configures GP18 for PWM and leaves the slice disabled. buzzer_play(freq_hz, duration_ms) computes the wrap value from $\text{clk_sys} / \text{freq_hz}$, applies 50% duty cycle, enables the slice, blocks for duration_ms, then disables it.

2.4 C Implementation – game.c

2.4.1 Welcome Banner and Difficulty Selection

On startup, print_welcome renders the GROUP 2 ASCII art banner and full instructions. The player then selects the difficulty by entering 'E' (one dot) or 'H' (four dots) in Morse code. The console output during startup and difficulty selection is shown below:

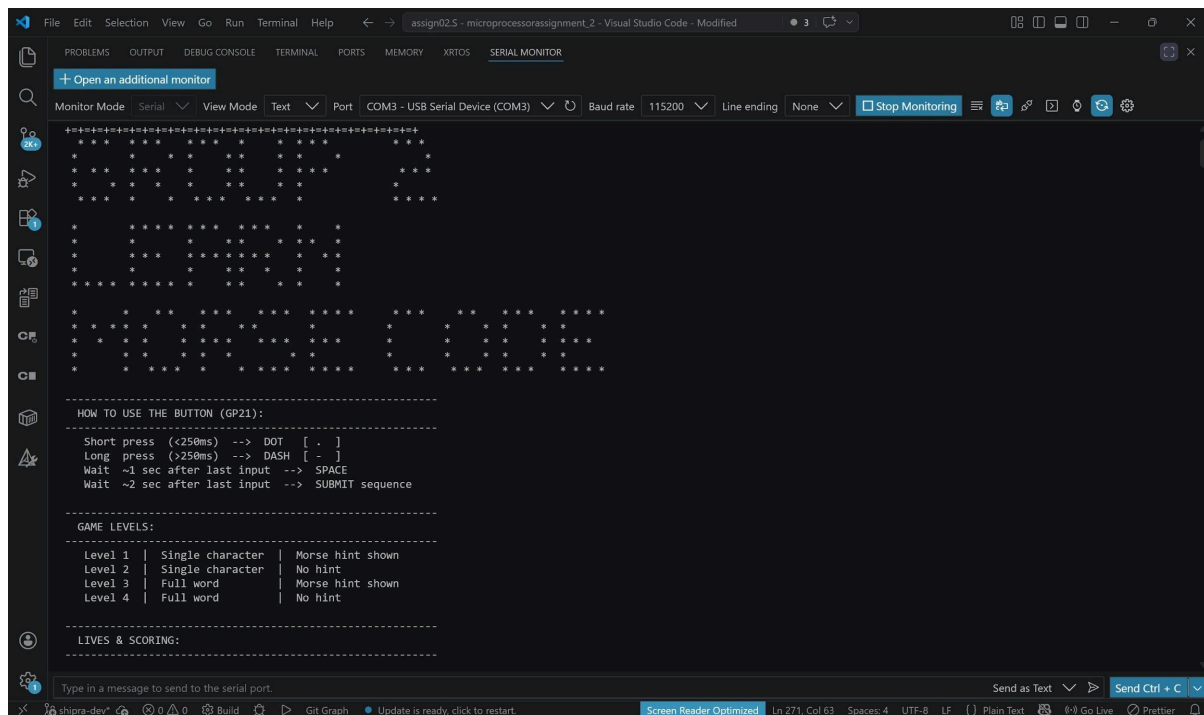


Figure 5 – Welcome banner with ASCII art, instructions, and button usage guide (VS Code Serial Monitor at 115200 baud)

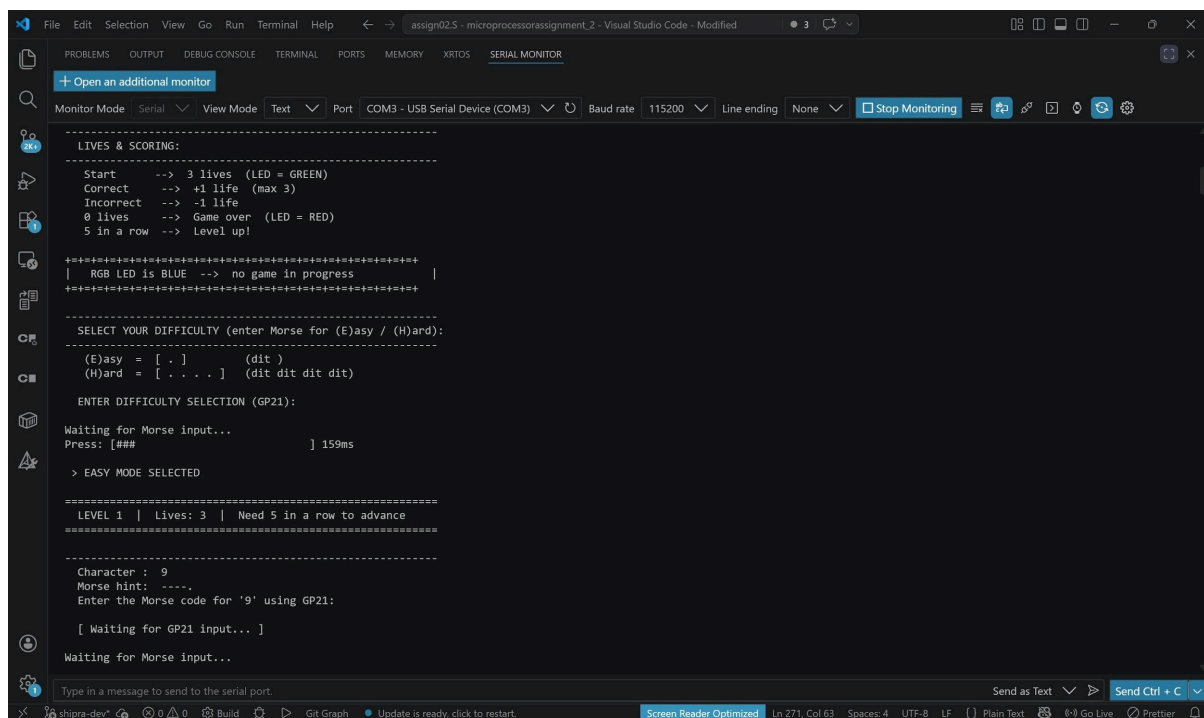


Figure 6 – Difficulty selection: player enters a short press (159 ms dot), Easy Mode selected, Level 1 begins with Character '9' and Morse hint '----'

2.4.2 wait_for_input and Visual Timeline (Feature #1)

The polling loop in `wait_for_input` drives the real-time press bar chart. Each '#' represents 50 ms of press duration. The screenshots below show the bar chart appearing for each button press, with the decoded result shown once the sequence is committed. Note the clear difference in bar width between short presses (dots, ~150–230 ms) and long presses (dashes, ~470–650 ms).

```

=====
LEVEL 1 | Lives: 3 | Need 5 in a row to advance
=====

Character : 9
Morse hint: ----.
Enter the Morse code for '9' using GP21:

[ Waiting for GP21 input... ]

Waiting for Morse input...
Press: [#####] 393ms
Press: [#####] 470ms
Press: [#####] 472ms

You entered : ---
Decoded as : 'O'
>> INCORRECT. '9' in Morse is: ----.
-----
Lives: 2 | Streak reset

Character : S
Morse hint: ...
Enter the Morse code for 'S' using GP21:

[ Waiting for GP21 input... ]

Waiting for Morse input...

You entered : -.
Decoded as : 'N'
>> INCORRECT. 'S' in Morse is: ...
-----
Lives: 1 | Streak reset
=====

```

Figure 7 – Level 1: three dashes entered (each ~390–472 ms bar), decoded as 'O', but target was '9' (----) - INCORRECT, lives drop to 2, then streak reset. Next character 'S' shown.

```

=====
Character : W
Morse hint: .--
Enter the Morse code for 'W' using GP21:

[ Waiting for GP21 input... ]

Waiting for Morse input...
Press: [####] 201ms
Press: [#####] 487ms
Press: [#####] 480ms

You entered : .--
Decoded as : 'W'
>> CORRECT! Great work!
-----
Lives: 3 | Streak: 2 / 5

Character : F
Morse hint: ...
Enter the Morse code for 'F' using GP21:

[ Waiting for GP21 input... ]

Waiting for Morse input...
Press: [####] 289ms
Press: [####] 290ms
Press: [#####] 645ms
Press: [####] 288ms

You entered : ...
Decoded as : '?'
>> INCORRECT. 'F' in Morse is: ...
-----
Lives: 2 | Streak reset
=====

```

Figure 8 – Level 1: '-.' entered for character 'W' (201 ms dot + two ~480 ms dashes = .-), CORRECT. Lives: 3, Streak: 2/5. Next character 'F' shown.

2.4.3 Level Progression and Streak System

Five consecutive correct answers complete a level. The level complete message shows final accuracy statistics. The screenshots below show a full level completion and the transition into Level 2 (no Morse hint):

```

File Edit Selection View Go Run Terminal Help
assign02.5 - microprocessorassignment_2 - Visual Studio Code - Modified
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS MEMORY XRTOS SERIAL MONITOR
+ Open an additional monitor
Monitor Mode Serial View Mode Text Port COM3 - USB Serial Device (COM3) Baud rate 115200 Line ending None Stop Monitoring
Character : Y
Morse hint: ----
Enter the Morse code for 'Y' using GP21:

[ Waiting for GP21 input... ]

Waiting for Morse input...
Press: [#####] 669ms
Press: [#####] 308ms
Press: [#####] 588ms
Press: [#####] 642ms

You entered : ----
Decoded as : '?'
>> INCORRECT. 'Y' in Morse is: ----

Lives: 2 | Streak reset

Character : H
Morse hint: ....
Enter the Morse code for 'H' using GP21:

[ Waiting for GP21 input... ]

Waiting for Morse input...
Press: [####] 219ms
Press: [####] 232ms
Press: [####] 237ms
Press: [###] 192ms

You entered : ....
Decoded as : 'H'
>> CORRECT! Great work!

Lives: 3 | Streak: 1 / 5

Type in a message to send to the serial port.
Send as Text Send Ctrl + C
shippa-dev Build Git Graph Update is ready, click to restart. Screen Reader Optimized Ln 271, Col 63 Spaces: 4 UTF-8 LF Plain Text Go Live Prettier

```

Figure 9 – Streak 1/5 achieved on 'H' (four short dots ~100–237 ms each =); Lives: 3. Feature #1 bar chart clearly visible for each press.

```

File Edit Selection View Go Run Terminal Help
assign02.5 - microprocessorassignment_2 - Visual Studio Code - Modified
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS MEMORY XRTOS SERIAL MONITOR
+ Open an additional monitor
Monitor Mode Serial View Mode Text Port COM3 - USB Serial Device (COM3) Baud rate 115200 Line ending None Stop Monitoring
Character : H
Morse hint: ....
Enter the Morse code for 'H' using GP21:

[ Waiting for GP21 input... ]

Waiting for Morse input...
Press: [###] 150ms
Press: [##] 115ms
Press: [##] 117ms
Press: [##] 100ms

You entered : ....
Decoded as : 'H'
>> CORRECT! Great work!

Lives: 3 | Streak: 5 / 5

** LEVEL 1 COMPLETE! **
Final score: 17 correct out of 25 (68%)

=====
LEVEL 2 | Lives: 3 | Need 5 in a row to advance
=====

Character : O
Morse hint: (hidden - Level 2)
Enter the Morse code for 'O' using GP21:

[ Waiting for GP21 input... ]

Waiting for Morse input...
Press: [#####] 535ms
Press: [#####] 532ms
Press: [#####] 514ms

You entered : ---

Type in a message to send to the serial port.
Send as Text Send Ctrl + C
shippa-dev Build Git Graph Update is ready, click to restart. Screen Reader Optimized Ln 271, Col 63 Spaces: 4 UTF-8 LF Plain Text Go Live Prettier

```

Figure 10 – LEVEL 1 COMPLETE: Streak 5/5 reached. Final score 17/25 (68%). Level 2 begins with Character 'O', Morse code hint hidden.

2.4.4 Level 2 – No Hint

In Level 2, the Morse hint is replaced with '(hidden - Level 2)'. The player must recall the code from memory. The bar chart (Feature #1) remains active, providing duration feedback even without the code hint:

```
=====
LEVEL 2 | Lives: 3 | Need 5 in a row to advance
=====

Character : O
Morse hint: (hidden - Level 2)
Enter the Morse code for 'O' using GP21:

[ Waiting for GP21 input... ]

Waiting for Morse input...
Press: [#####] 535ms
Press: [#####] 532ms
Press: [#####] 514ms

You entered : ---
Decoded as : 'O'
>> CORRECT! Great work!

Lives: 3 | Streak: 1 / 5

-----

Character : B
Morse hint: (hidden - Level 2)
Enter the Morse code for 'B' using GP21:

[ Waiting for GP21 input... ]

Waiting for Morse input...
Press: [#####] 601ms
Press: [####] 222ms
Press: [####] 226ms
Press: [####] 219ms

You entered : -...
Decoded as : 'B'
>> CORRECT! Great work!
```

Figure 11 – Level 2: 'O' entered correctly (three dashes ~514–535 ms each = ---), CORRECT. Then 'B' entered correctly (-... = one long + three short). Feature #1: bar chart visible throughout.

2.4.5 Level 2 Complete → Level 3 (Word Level)

After completing Level 2, the game transitions to Level 3, where full words must be entered letter by letter. The Morse sequence for the entire word is provided as a hint. The underscore progress bar fills in as each letter is decoded correctly:

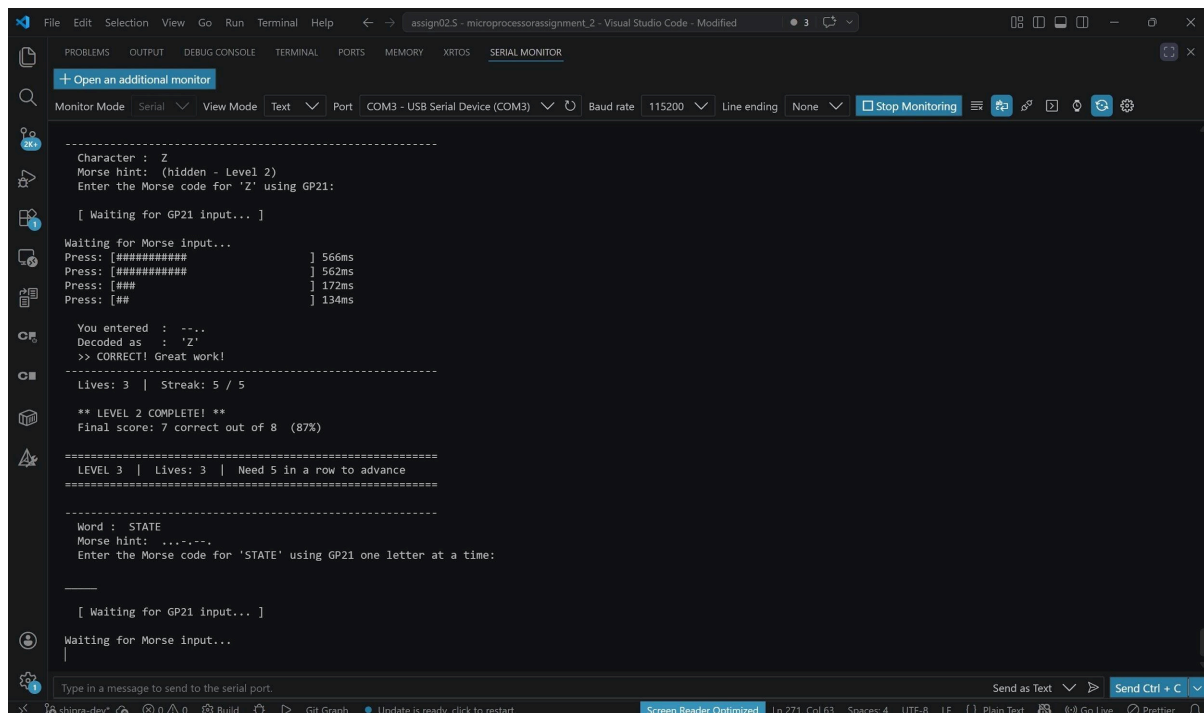


Figure 12 – LEVEL 2 COMPLETE: 7/8 correct (87%). Level 3 begins with the word 'STATE', Morse hint '---.-.-.' shown. Underscore progress bar '_____' displayed. The player is waiting to enter the first letter.

2.4.6 Level 3 – Word with Morse Hint

Level 3 requires the player to enter Morse code for each letter of a complete word, one at a time. The word and its full Morse sequence are shown as a hint. A progress display of underscores fills in left-to-right as each letter is decoded correctly:

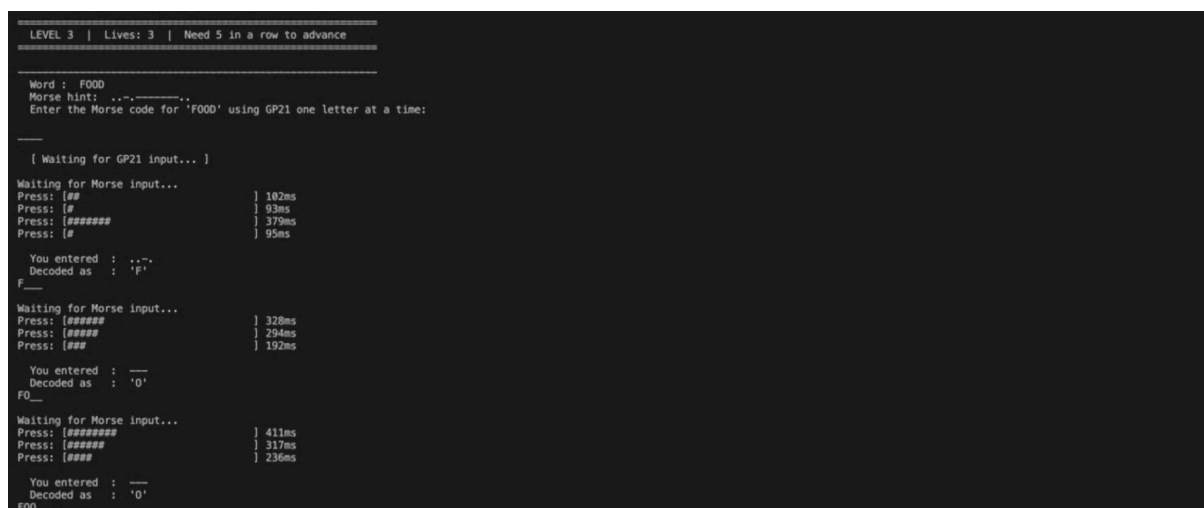


Figure 13 – Level 3: word 'FOOD' with Morse hint shown. Player enters F(..-.), then O(---) twice. Progress bar fills: 'F__' → 'FO__' → 'FOO_'. Feature #1: The bar chart is visible with every press.

2.4.7 Level 4 – Word without Morse Hint

Level 4 is the hardest mode: a full word must be entered letter-by-letter with no Morse hint displayed. The player must recall the correct sequence for each letter entirely from memory. The progress bar still updates after each decoded letter:

```
=====
LEVEL 4 | Lives: 3 | Need 5 in a row to advance
=====

Word: RED
Morse hint: (hidden - Level 4)
Enter the Morse code for 'RED' using GP21 one letter at a time:

[ Waiting for GP21 input... ]

Waiting for Morse input...
Press: [# ] 139ms
Press: [#####] 355ms
Press: [# ] 98ms

You entered : -.
Decoded as : 'R'
R_

Waiting for Morse input...
Press: [# ] 128ms

You entered : .
Decoded as : 'E'
RE_

Waiting for Morse input...
Press: [####] 332ms
Press: [# ] 84ms
Press: [# ] 87ms

You entered : -..
Decoded as : 'D'
RED
>> CORRECT! Great work!

Lives: 3 | Streak: 1 / 5
```

Figure 14 – Level 4: word 'RED', hint hidden. Player correctly enters R (.-), E (.), D (-..). Progress fills: 'R__' → 'RE_' → 'RED'. CORRECT! Streak: 1/5.

2.5 welcome.c – Startup Banner

print_welcome uses print_slow (per-character output with configurable delay) and print_fast (direct printf) to display a TCD header, GROUP 2, and LEARN MORSE CODE ASCII art, and structured instruction sections. The typing effect gives players time to read instructions before any input is required, as seen in Figure 5.

3. Innovative Features

IMPORTANT: Both innovative features must be clearly identified in this report and demonstrated in the video. Both features are fully implemented, integrated into gameplay, and explicitly documented below with the exact source file and function where each is implemented.

INNOVATIVE FEATURE #1 - Console Visual Press-Duration Timeline

What it does: Every time the GP21 button is released, the C polling loop reads the press_duration value (exported as .global from the ARM assembly .data section) and prints a proportional bar chart to the serial console. Each '#' character = 50 ms of press duration, bar width capped at 30. The raw millisecond value is shown to the right. Once the full Morse symbol is submitted, the decoded alphanumeric character is printed.

Where in the code: game.c → wait_for_input() + assign02.S → .global press_duration

Why it is innovative: The ARM interrupt handler (gpio_isr) is not modified for this feature - it only stores the raw duration in press_duration. All bar rendering and output is in the C main loop in wait_for_input(), satisfying the assignment's explicit constraint. The feature makes the

invisible dot/dash threshold directly visible to the player, enabling self-correction and accelerating learning through feedback on every single press.

How to identify in the video: Look for 'Press: [####...] Xms' lines appearing on the console immediately after each button release (clearly visible in Figures 7–12 throughout the report). The bar grows proportionally with press duration - short presses for dots, long presses for dashes.

Feature #1 is clearly visible in every console screenshot (Figures 7–12). Example output:

```
Waiting for Morse input...
Press: [####          ] 201ms ← short press = dot
Press: [#####        ] 487ms ← long press = dash
Press: [#####        ] 480ms ← long press = dash

You entered : .--
Decoded as  : 'W'
>> CORRECT! Great work!
```

INNOVATIVE FEATURE #2 - PWM-Based Buzzer Feedback

What it does: After every round, the buzzer on GP18 plays the correct Morse pattern for the target. A dot = 150 ms tone, a dash = 450 ms tone, with 150 ms gaps between symbols. Frequency is 2400 Hz for a correct answer and 400 Hz for an incorrect one, giving simultaneous auditory feedback on both the Morse pattern and the answer outcome.

Where in the code: assign02.c → buzzer_init(), buzzer_play() | game.c → buzz_morse_sequence()

Why it is innovative: The ARM ISR is not modified for this feature. buzzer_play() uses the RP2040 hardware PWM peripheral: computes wrap value from $\text{clk_sys} / \text{freq_hz}$, sets 50% duty cycle, enables the slice, blocks for the tone duration, then disables it. The C layer reads the already-classified dot/dash information from the Morse lookup string and drives the PWM hardware independently. The dual-frequency scheme (bright 2400 Hz correct, dull 400 Hz incorrect) makes both the Morse pattern and the answer result audible simultaneously, reducing reliance on the visual console.

How to identify in the video: Listen for a tone playing after each input is submitted. High-pitched tone (2400 Hz) after a correct answer; low-pitched (400 Hz) after an incorrect. Dot tones are short (~150 ms), dash tones are longer (~450 ms). It should be clearly audible in the demonstration video throughout all four levels.

The buzzer implementation in assign02.c:

```
void buzzer_play(uint32_t freq_hz, uint32_t duration_ms) {
    uint32_t top = clock_get_hz(clk_sys) / freq_hz - 1;
    pwm_set_wrap(slice, top);
    pwm_set_chan_level(slice, chan, top / 2); // 50% duty cycle
    pwm_set_enabled(slice, true);
    sleep_ms(duration_ms);
    pwm_set_enabled(slice, false);
}
```

Called from game.c in buzz_morse_sequence() - plays correct Morse pattern regardless of whether answer was right or wrong:

```
for (int i = 0; morse[i] != '\0'; i++) {  
    if (morse[i] == '.') buzzer_play(freq, 150); // dot = 150 ms  
    if (morse[i] == '-') buzzer_play(freq, 450); // dash = 450 ms  
    if (morse[i+1] != '\0') sleep_ms(150); // inter-symbol gap  
}
```

4. Application Demo

This section provides an overview of the hardware demonstration and game completion flow, supplementing the video submission (D5).

4.1 Game Completion

The screenshot below shows the final output when a player completes all four levels. Level 4 finishes with a score of 6 correct out of 7 (85%), followed immediately by the game completion congratulations message:

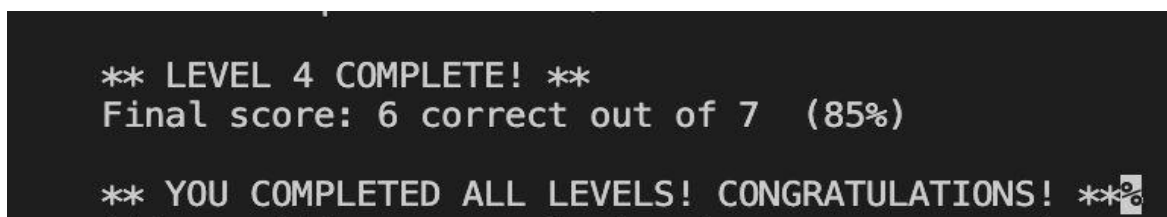


Figure 15 – Game completion: LEVEL 4 COMPLETE (6/7, 85%) followed by '** YOU COMPLETED ALL LEVELS! CONGRATULATIONS! **'

4.2 Hardware LED Summary

The four RGB LED states on the physical MAKER-PI-PICO board (GP28, WS2812 PIO-driven) are shown in Figures 1-4. The mapping is summarised below:

State	LED Colour	GRB Value	When Active
Idle / No game	Blue	0x00001F	At startup, before difficulty is selected
3 lives (full health)	Green	0x1F0000	After the correct answer or at the start
2 lives	Yellow	0x1F1F00	After the first incorrect answer
1 life	Red	0x001F00	After the second incorrect answer

5. Conclusion

The Group 2 Morse Code Game fully satisfies all seven core requirements of Assignment #02. All four levels operate correctly as demonstrated in Figures 5-15 throughout this report, with hardware LED states shown in Figures 1-4. The codebase maintains a clean boundary between ARM assembly (GPIO timing, ISR, Morse buffer in assign02.S) and C (game logic, LED, and buzzer in assign02.c and game.c).

Both innovative features are fully implemented and integrated into gameplay. Feature #1 (console press-duration bar chart) is visible in every console screenshot. Feature #2 (PWM buzzer) plays the correct Morse pattern after every input using RP2040 hardware PWM, with dual frequencies distinguishing correct from incorrect answers - audible in the demonstration video.

The team managed the project with defined roles, three tracked milestones, and an active GitLab feature-branch workflow, producing approximately 50 commits from all members across the five weeks.

6. Console Output Index

The table below maps every figure in this report to the game state it illustrates, providing a complete walkthrough from startup to game completion:

Figure	Level / State	Key Feature Shown
Fig. 5	Startup	Welcome banner, GROUP 2 ASCII art, instructions
Fig. 6	Difficulty selection → Level 1	Easy Mode via dot (159 ms). Feature #1 bar.
Fig. 7	Level 1 – incorrect round	Three dashes decoded as 'O ', not '9'. Lives: 2.
Fig. 8	Level 1 – correct round	'W' (.-) entered correctly. Streak 2/5.
Fig. 9	Level 1 – streak building	Four dots for 'H'. Streak 1/5.
Fig. 10	Level 1 complete → Level 2	Score 17/25 (68%). Level 2 begins, hint hidden.
Fig. 11	Level 2 – no hint rounds	'O' and 'B' from memory. Feature #1 bar visible.
Fig. 12	Level 2 complete → Level 3	Score 7/8 (87%). Level 3: word 'STATE', hint shown.
Fig. 13	Level 3 – word with a hint	'FOOD' progress: F__ → FO__ → FOO_ → FOOD.
Fig. 14	Level 4 – word no hint	'RED' from memory. R→E→D. Streak 1/5.
Fig. 15	Game complete	Level 4: 6/7 (85%). Congratulations message.

7. References

- Raspberry Pi Ltd. RP2040 Datasheet, 2021. Available: <https://datasheets.raspberrypi.com/rp2040/rp2040-datasheet.pdf>
- Raspberry Pi Ltd. Raspberry Pi Pico C/C++ SDK, 2021. Available: <https://datasheets.raspberrypi.com/pico/raspberry-pi-pico-c-sdk.pdf>
- ARM Ltd. Cortex-M0+ Technical Reference Manual (DDI0484). Available: <https://developer.arm.com/documentation/ddi0484/latest>
- ITU-R Recommendation M.1677-1: International Morse Code. International Telecommunication Union, 2009.
- Worldsemi Co. WS2812B Datasheet. Available: <https://cdn-shop.adafruit.com/datasheets/WS2812B.pdf>
- CSU23021 Lecture Slides and Lab Handouts, School of Computer Science and Statistics, TCD, 2025/2026.